

Phoenix Cross Dock WMS

Platform Rebuild & Go-Live Plan

1. Why We're Rebuilding Before Going Live

The application currently exists as a single HTML file running entirely in the browser, with no server and no database. In practice this means nothing is actually saved — every pallet, invoice, customer, and shipment record lives only in the browser's temporary memory and disappears the moment the page is refreshed or closed. This was fine for building out the look and feel and workflow, but it cannot safely run as a live, multi-user system.

It also cannot support what you've asked for next. A QuickBooks integration requires securely storing login credentials and exchanging data with QuickBooks' servers — work that has to happen on a protected backend server, not in a public webpage where anyone could view the underlying code. The rebuild gives the application the three things it's currently missing:

- A real database, so records persist and multiple staff can work at the same time without losing data.
- A secure backend server to safely handle QuickBooks (and future integrations) without exposing credentials.
- A proper user login system, so different staff/customers only see what they should.

2. What We're Building It With

Frontend	Next.js + TypeScript	Rebuilds the existing screens (Dashboard, Pallets, Invoices, Customers, etc.) with the same look and workflow you already approved, on a modern, maintainable foundation.
Backend	Express.js + TypeScript	A secure API server that handles business logic and is the only place that ever talks to QuickBooks or other outside services.
Database	MongoDB	Permanent storage for customers, pallets, invoices, and warehouse data — the piece that doesn't exist today.

3. Source Control — GitHub

We'll set up a private GitHub repository (under your account/organization, so you own the code, not us). Every change we make is committed there with a full history, which means we can always see what changed, roll back a bad change instantly, and later add automatic testing/deployment on top of it.

4. Deployment — AWS (EC2)

The rebuilt app will be deployed on an Amazon Web Services EC2 server — a private virtual server we configure and control (rather than a shared hosting plan), which is what's needed to run the backend, database connection, and QuickBooks integration continuously and securely.

- Provision the EC2 server and connect it to the GitHub repository for deployment.
- Install and run the Next.js frontend and Express.js backend on the server.
- Point your domain at the server and enable HTTPS (SSL) so the site is secure.
- Store QuickBooks and other credentials as protected server secrets — never in the code itself.

5. Rollout Plan

Phase	What Happens
1. Rebuild	Recreate current features (Dashboard, Pallets, LPNs, Invoices, Customers, Requests, Warehouse Setup) on the new stack with a real database.
2. Verify	Confirm the rebuilt app matches current functionality before adding anything new.
3. Integrate	Connect QuickBooks and any other requested integrations on the secure backend.
4. Deploy	Launch on AWS, connect the domain, enable SSL, go live.
5. Handover	Final testing, walkthrough, and access handover to you.

Questions on any part of this plan? Happy to walk through it together before we start.

6. Timeline & Go-Live Schedule

Target: kickoff Thursday, September 24, 2026, with the platform delivered before October 1, 2026, working weekdays only (no Saturday/Sunday work).

Date	Phase	Focus
Thu, Sep 24	Kickoff	Repo setup, environment config, EC2 provisioning starts, MongoDB schema design, auth scaffolding.
Fri, Sep 25	1. Rebuild	Backend API build-out: core endpoints for Pallets, LPNs, Invoices, Customers, Requests, Warehouse Setup.
Mon, Sep 28	1. Rebuild	Frontend rebuild: Dashboard, Pallets, LPNs, Invoices, Customers, Requests, Warehouse Setup — wired to the new backend and login system.
Tue, Sep 29	2. Verify + 3. Integrate	Feature-by-feature comparison against the current app and bug fixes; QuickBooks connection on the secure backend.
Wed, Sep 30	4. Deploy + 5. Handover	EC2 deployment, domain + SSL, credentials stored as protected server secrets; final testing, client walkthrough, access handover. Delivery completed before October.

Risk note on the compression:

Excluding weekends leaves only five working days (Sep 24, 25, 28, 29, 30) before October to fit the full scope — rebuild, verification, QuickBooks integration, and AWS production deployment. That does not leave room for the requested 2–3 day margin; Sep 30 functions as same-day buffer and handover rather than a true multi-day cushion. Tuesday Sep 29 and Wednesday Sep 30 are the highest-risk days: QuickBooks integration depends on external API access, and deployment/SSL setup depends on DNS propagation, which isn't fully in our control. If QuickBooks developer credentials aren't already provisioned, requesting them now is the single biggest thing that protects this timeline.

Recommended fallback: if Sep 29–30 slips, the core rebuild (Phases 1–2, done by Mon Sep 28) can still go live as a standalone milestone on schedule, with QuickBooks integration and production deployment following as a short fast-follow in the first weekdays of October — which also restores a genuine 2–3 day margin on the piece most likely to need it.